

```
In [9]: import pandas as pd
arr=pd.Series([1,2,3,4,56],index=['A','B','C','D','E'])
print(arr)
print(arr['E'])
#vectorized operation no loop required
print(arr+5)
print(arr.iloc[1])
print(arr.loc['A'])
print(arr.sum())
print(arr.std())
print(arr.max())
print(arr.min())
print(arr.mean())
print(arr.median())
print(arr.var())

#Boolean Indexing
print(arr[arr>30])

#count values
print(len(arr))
# it return individual value count
print(arr.value_counts())

#string operation
name = pd.Series(['kavya','mithun','riya'])
print(name.str.upper())
print(name.str.len())
```

```

A      1
B      2
C      3
D      4
E     56
dtype: int64
56
A      6
B      7
C      8
D      9
E     61
dtype: int64
2
1
66
23.95203540411545
56
1
13.2
3.0
573.6999999999999
E     56
dtype: int64
5
1      1
2      1
3      1
4      1
56     1
Name: count, dtype: int64
0      KAVYA
1     MITHUN
2      RIYA
dtype: object
0      5
1      6
2      4
dtype: int64

```

In []:

```

In [7]: import pandas as pd
a = pd.Series(['divi','arthi','priya','moli'])
print(a)
print(a.values)
print(a.index)
print(a.dtype)

#accessing elements
print(a[1])

#accessing element using loc
print(a.loc[2])
print(a.iloc[2])

```

```
#slicing
print(a[1:3])
print(a.loc[1:3])
print(a.iloc[1:3])
```

```
0    divi
1    arthi
2    priya
3     moli
dtype: object
['divi' 'arthi' 'priya' 'moli']
RangeIndex(start=0, stop=4, step=1)
object
arthi
priya
priya
1    arthi
2    priya
dtype: object
1    arthi
2    priya
3     moli
dtype: object
1    arthi
2    priya
dtype: object
```

```
In [11]: import pandas as pd
a = pd.Series({
    "gayu":16,
    "rithi":22,
    "ramya":28,})
print(a)
print(a["gayu"])
```

```
gayu    16
rithi    22
ramya    28
dtype: int64
16
```

```
In [12]: #NaN - float type value
#index alignment
s1 = pd.Series([1,2,3,4,5])
s2 = pd.Series([1,2,3],index=['B','C','d'])
print(s1+s2)

s = pd.Series([1,2,3,4,5])
print(s.count())
print(s.unique())
print(s.nunique())
```

```
0    NaN
1    NaN
2    NaN
3    NaN
4    NaN
B    NaN
C    NaN
d    NaN
dtype: float64
5
[1 2 3 4 5]
5
```

data frame

```
In [35]: import pandas as pd
#method 1 : From Dictionary
data = ({
    'name':['arathi','priya','divi'],
    'roll_no':[4,3,8],
    'age':[21,28,23,]})
df = pd.DataFrame(data)
print(df)

#method 2 : From series to DataFrame
s1 = pd.Series([1,2,3])
s2 = pd.Series([4.9,5.4,6.3])
s3 = pd.Series([True,False,True])
df = pd.DataFrame({'sno' : s1,
                   'cgpa' : s2,
                   'attendance' : s3})

print(df)
#list of list
data =[['dharshini',22,98],
       ['dhara',21,88],
       ['pavi',20,77]]
df=pd.DataFrame(data,columns=["name","age","mark"])
print(df)

#list of Dictionary
data = [
    {"name": "bhuvi","course": "cs","grade":"A"},
    {"name": "vino","course":"bca","grade":"A++"},
    {"name": "subha","course":"aiml","grade":"A"}]
df = pd.DataFrame(data,index = ["std1","std2","std3"])
print(df)
#Adding one extra columns
df['marks']=[99,66,56]
print(df)

#Adding multiple rows
std4 = pd.DataFrame([{"name":"kathir","course":"cs","grade":"A+" , "marks" :90}],in
df = pd.concat([df,std4])
print(df)
```

```
#create multiple rows
students = pd.DataFrame([{"name":"rani","course":"ds","grade":"A+","marks": 90},
                          {"name":"thara","course":"csai","grade":"A","marks":99}],i
df = pd.concat([df,students])
print(df)
```

```
   name  roll_no  age
0  arthi         4   21
1  priya         3   28
2  divi         8   23
   sno  cgpa  attendance
0    1   4.9          True
1    2   5.4          False
2    3   6.3          True
   name  age  mark
0  dharshini  22   98
1    dhara   21   88
2    pavi   20   77
   name  course  grade
std1  bhuvi    cs    A
std2  vino    bca   A++
std3  subha  aiml    A
   name  course  grade  marks
std1  bhuvi    cs    A    99
std2  vino    bca   A++   66
std3  subha  aiml    A    56
   name  course  grade  marks
std1  bhuvi    cs    A    99
std2  vino    bca   A++   66
std3  subha  aiml    A    56
std4  kathir    cs   A+   90
   name  course  grade  marks
std1  bhuvi    cs    A    99
std2  vino    bca   A++   66
std3  subha  aiml    A    56
std4  kathir    cs   A+   90
std5   rani     ds   A+   90
std6  thara  csai    A    99
```

In []: *#Column Access by Bracket Notation and Dot Notation*

```
print(df['marks'])
print(df.course)
print(df[['grade','marks']])
```

```
#shape attribute
print(df.shape)
```

```
#Find Number Of Rows
print(df.shape[0])
```

```
#Find Number Of Columns
print(df.shape[1])
```

```
#Dtype Attribute
```

```

#Used to check Datatype of all columns
print(df.dtypes)

#Info
print(df.info())

print(df.describe())

#Select Specific Rows
print(df.loc["std3"])

#Update Value
df.loc['std1','grade']='B++'
print(df)
df.loc['std1','name']='Surya'
print(df)

#Delete Column
df.drop('grade',axis=1,inplace=True)
print(df)

#Delete Row
df.drop('std6',axis=0,inplace=True)
print(df)

#Sort Values
df.sort_values('marks',inplace=True)
print(df)

#Decending
df.sort_values('marks',ascending = False, inplace=True)
print(df)

#Statistical Function for particular Column
print(df['marks'].max())
print(df['marks'].min())
print(df['marks'].mean())
print(df['marks'].median())
print(df['marks'].std())
print(df['marks'].mode())
print(df['marks'].var())

```

In []:

In []: